

Министерство науки и высшего образования Российской Федерации
Пензенской государственный университет
Кафедра "Вычислительная техника"

Пояснительная записка

К курсовому проектированию
по курсу: «Логика и основы алгоритмизации в инженерных задачах»
на тему: «Реализация алгоритма выделения компонент связности орграфа,
используя поиск в ширину»

Выполнил студент группы

24BBB1:

Путютин С.А.

Принял:

к.т.н., доцент Юрова О.В.

28.12.25
Хорошо
Юрова

Пенза 2025

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Вычислительной техники

Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

«__» __ 20__

ЗАДАНИЕ

на курсовое проектирование по курсу

Логика и основы алгоритмизации в машинных задачах
Студенту Пустырицу Сергею Алексеевичу Группа 24 ВВВ1
Тема проекта Валидация алгоритма вычислений компонент
связности графов, используя поиск в ширину

Исходные данные (технические требования) на проектирование

Разработка алгоритмов и программного обеспечения в соответствии с данными заданиями курсового проекта. Проектная записка должна содержать:

1. Постановку задачи;
2. Техническую часть задания;
3. Описание алгоритма поставленной задачи;
4. Пример ручного расчета задачи с использованием алгоритма;
5. Описание самой программы;
6. Тесты;
7. Список литературы;
8. Листинг программы;
9. Результаты работы программы.

Объем работы по курсу

1. Расчетная часть

Бумажный расчет работы алгоритма

2. Графическая часть

Схема алгоритма в форме блок-схем

3. Экспериментальная часть

Тестирование программы;
Результаты работы программы на тестовых данных

Срок выполнения проекта по разделам

- 1 Исследование теоретической части курсового проекта
- 2 Разработка алгоритмов программы
- 3 Разработка программы
- 4 Тестирование и завершение разработки программы
- 5 Оформление письменной записки
- 6
- 7
- 8

Дата выдачи задания "12" сентября

Дата защиты проекта " " "

Руководитель Курова О.В. Федя

Задание получил "02" октября 2025 г.

Студент Путятин С.А. Отм

Содержание

Реферат	5
Введение	6
1. Постановка задачи.....	7
2. Теоретическая часть задания.....	8
3.Описание алгоритма	9
4.Описание программы.....	12
5.Тестирование	18
6.Ручной расчёт задачи	22
Заключение.....	24
Список литературы	25
ПриложениеА.	26

Реферат

Отчет 40 стр., 8 рисунков, 1 таблица.

ГРАФ, ТЕОРИЯ ГРАФОВ, ОРГРАФ, ДОСТИЖИМОСТЬ, ПОИСК В ШИРИНУ, КОМПОНЕНТА СВЯЗНОСТИ, BFS.

Цель исследования – разработка программы, способной выделять компоненты связности ориентированного графа (орграфа) с использованием алгоритма поиска в ширину (BFS).

В работе рассмотрены принципы поиска в ширину, на основе которых реализуется алгоритм выделения компонент связности орграфа. Установлено, что с помощью данного алгоритма можно эффективно находить все компоненты связности орграфа, независимо от степени его связности (сильно связный, слабо связный или несвязный).

Введение

Алгоритм поиска в ширину (англ. breadth-firstsearch, BFS) – один из основных алгоритмов обхода графов. В отличие от поиска в глубину (DFS), который исследует граф "вглубь", BFS исследует граф "вширь", обрабатывая сначала все вершины, находящиеся на расстоянии одного ребра от начальной вершины, затем на расстоянии двух ребер и так далее.

Основные особенности BFS:

- 1.Находит кратчайшие пути в невзвешенных графах
- 2.Обрабатывает вершины в порядке увеличения расстояния от начальной вершины
- 3.Использует очередь для хранения вершин, которые нужно обработать
- 4.Гарантированно находит все достижимые вершины из начальной точки

В контексте ориентированных графов BFS применяется для определения достижимости вершин и выделения компонент связности. Компонента связности орграфа – это максимальное по включению множество вершин, в котором для любой пары вершин существует путь (в случае сильной связности – в обе стороны).

Цель работы: разработать программу на языке Си, реализующую алгоритм поиска в ширину для выделения компонент связности орграфа. Программа должна предоставлять различные способы задания графа, включая генерацию случайных графов и работу с предопределенными примерами.

1. Постановка задачи

Требуется разработать программу, которая выделяет компоненты связности орграфа, используя алгоритм поиска в ширину.

Технические требования:

Ввод данных:

1. Пользователь вводит количество вершин графа
2. Возможность генерации случайной матрицы смежности с заданной вероятностью присутствия ребра
3. Возможность выбора из предопределенных графов (сильно связный, слабо связный, несвязный)

Обработка данных:

1. Реализация алгоритма BFS для обхода графа
2. Выделение всех компонент связности
3. Определение типа связности орграфа (сильно связный, слабо связный, несвязный)

Вывод результатов:

1. Отображение матрицы смежности
2. Определение и вывод типа связности орграфа
3. Вывод всех найденных компонент связности с номерами вершин

Дополнительные функции:

1. Сохранение графа в файл
2. Загрузка графа из файла
3. Визуализация графа в формате DOT для Graphviz

2. Теоретическая часть задания

Ориентированный граф (орграф) – это пара $G = (V, E)$, где:

V – множество вершин $E \subseteq V \times V$ – множество ориентированных ребер (дуг).

Матрица смежности – квадратная матрица размером $|V| \times |V|$, где элемент $a[i][j] = 1$, если существует ребро из вершины i в вершину j , и $a[i][j] = 0$ в противном случае.

В ориентированных графах различают несколько типов связности:

1. Сильная связность: орграф называется сильно связным, если для любой пары вершин u и v существует путь из u в v и путь из v в u .

2. Слабая связность: орграф называется слабо связным, если неориентированный граф, полученный заменой всех ориентированных ребер на неориентированные, является связным.

3. Компоненты сильной связности: максимальные сильно связные подграфы орграфа.

Принцип работы BFS:

1. Начинаем с начальной вершины s
2. Помечаем s как посещенную
3. Добавляем s в очередь
4. Пока очередь не пуста:
5. Извлекаем вершину v из очереди
6. Для каждой непосещенной вершины u , смежной с v :
7. Помечаем u как посещенную
8. Добавляем u в очередь

3.Описание алгоритма

Для программной реализации алгоритма выделения компонент связности орграфа с использованием поиска в ширину понадобятся следующие структуры данных:

Структура Graph - для хранения матрицы смежности орграфа и количества вершин. Матрица смежности представляет собой квадратную матрицу размером $\text{MAX_VERTICES} \times \text{MAX_VERTICES}$, где элемент $\text{adj}[i][j] = 1$, если существует ориентированное ребро из вершины i в вершину j , и $\text{adj}[i][j] = 0$ в противном случае.

Структура Queue - для реализации очереди, необходимой для алгоритма поиска в ширину (BFS). Очередь содержит массив `items` для хранения элементов, а также индексы `front` и `rear` для управления доступом к элементам.

Массив `visited` - логический массив для хранения информации о посещенных вершинах (`true` - посещена, `false` - не посещена).

Основной алгоритм BFS:

Имеется граф $G = (V, E)$, где V - множество вершин, E - множество ориентированных ребер. Каждая из вершин изначально отмечена как не посещенная.

В качестве исходного пункта выбирается вершина s , и ей приписывается, что она посещена: $\text{visited}[s] = \text{true}$. Вершина s добавляется в очередь.

Пока очередь не пуста, извлекается вершина v из начала очереди. Для каждой вершины u , в которую ведет ребро из v ($\text{adj}[v][u] == 1$), и которая еще не посещена, выполняется: помечаем u как посещенную и добавляем u в конец очереди.

Таким образом, алгоритм обходит все вершины, достижимые из начальной вершины s , в порядке увеличения расстояния от s .

Алгоритм выделения всех компонент связности

Для нахождения всех компонент связности орграфа используется следующий алгоритм:

1. Инициализировать массив `visited`, установив все элементы в `false`.
2. Инициализировать счетчик компонент `componentCount = 0`.
3. Для каждой вершины i от 0 до $n-1$:
4. Если вершина i не посещена (`visited[i] == false`):
5. Вывести "Компонента X:", где $X = ++componentCount$
6. Инициализировать очередь
7. Пометить вершину i как посещенную
8. Добавить i в очередь
9. Пока очередь не пуста:
10. Извлечь вершину `currentVertex` из очереди
11. Вывести `currentVertex`
12. Для каждой вершины j от 0 до $n-1$:
13. Если существует ребро из `currentVertex` в j (`adj[currentVertex][j] == 1`) и вершина j не посещена:
14. Пометить j как посещенную
15. Добавить j в очередь
16. Вывести перевод строки
17. Вывести "Всего компонент связности: `componentCount`"

Псевдокод функции `BFSFromVertex`:

`BFSFromVertex`(граф g , вершина `startVertex`, массив `visited`):

1. Инициализировать очередь q
2. Вывести "Порядок обхода BFS начиная с вершины `startVertex`: "
3. `visited[startVertex] = true`
4. Добавить `startVertex` в очередь q

5. Пока очередь q не пуста:

5.1. currentVertex = извлечь вершину из очереди q

5.2. Вывести currentVertex

5.3. Для i от 0 до g.vertices-1:

5.3.1. Если g.adj[currentVertex][i] == 1 и visited[i] == false:

5.3.1.1. visited[i] = true

5.3.1.2. Добавить i в очередь q

6. Вывести перевод строки

Псевдокод функции findAllComponents:

findAllComponents(графg):

1. Инициализировать массив visited размером g.vertices значением false

2. componentCount = 0

3. Вывести "=== Все компоненты связности орграфа ==="

4. Для i от 0 до g.vertices-1:

4.1. Если visited[i] == false:

4.1.1. Вывести "Компонента " + (++componentCount) + ": "

4.1.2. Инициализировать очередь q

4.1.3. visited[i] = true

4.1.4. Добавить i в очередь q

4.1.5. Пока очередь q не пуста:

4.1.5.1. currentVertex = извлечь вершину из очереди q

4.1.5.2. Вывести currentVertex

4.1.5.3. Для j от 0 до g.vertices-1:

4.1.5.3.1. Если g.adj[currentVertex][j] == 1 и visited[j] == false:

4.1.5.3.1.1. visited[j] = true

4.1.5.3.1.2. Добавить j в очередь q

4.1.6. Вывести перевод строки

5. Вывести "Всего компонент связности: " + componentCount

4.Описание программы

Архитектура программы:

Программа разработана на языке Си и состоит из следующих модулей:

- 1.Основной модуль (main): управление программой, обработка меню
- 2.Модуль работы с графами: операции с матрицами смежности
- 3.Модуль алгоритмов BFS: реализация поиска в ширину
- 4.Модуль компонент связности: выделение и обработка компонент
- 5.Модуль работы с файлами: сохранение и загрузка графов
- 6.Модуль визуализации: экспорт в DOT формат

Пользовательский интерфейс:

1.Программа использует консольный интерфейс с текстовым меню. Все сообщения и инструкции выводятся на русском языке.

Основные экраны программы:

- 1.Главное меню: предоставляет доступ ко всем функциям программы
- 2.Экран генерации графа: настройка параметров случайного графа
- 3.Экран матрицы смежности: отображение структуры графа
- 4.Экран результатов BFS: показ обхода графа из заданной вершины
- 5.Экран компонент связности: отображение всех найденных компонент
- 6.Экран редактирования: ручное добавление/удаление ребер

Пример работы программы:

=== РЕАЛИЗАЦИЯ АЛГОРИТМА BFS ДЛЯ ОРГРАФА ===

Главное меню:

1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф

6. Сохранить граф в файл 'Graf1.txt'
7. Загрузить граф из файла 'Graf1.txt'
8. Выход

Выберите действие: 1

Введите количество вершин (1-50): 6

Матрица смежности:

v0 v1 v2 v3 v4 v5

v0: 0 1 0 0 1 0

v1: 0 0 1 0 0 0

v2: 1 0 0 0 0 0

v3: 0 0 0 1 0 0

v4: 0 0 0 0 0 1

v5: 0 0 1 0 0 0

Выберите действие: 4

=== КОМПОНЕНТЫ СВЯЗНОСТИ ОРГРАФА ===

Орграф слабо связный

Компонента 1: 0 1 2

Компонента 2: 3 4 5

Всего компонент: 2

Псевдокод функции main:

main():

1. Установить русскую локаль
2. Инициализировать генератор случайных чисел
3. graphInitialized = false
4. Попытаться загрузить граф из файла "Graf1.txt"
5. Если загрузка успешна: graphInitialized = true
6. Вывести заголовок программы
7. Цикл выполнять пока true:
 - 7.1. Отобразить меню

- 7.2. Считать choice от пользователя
- 7.3. Если choice == 1:
 - 7.3.1. Запросить количество вершин vertices
 - 7.3.2. Если vertices корректно:
 - 7.3.2.1. Сгенерировать случайный граф
 - 7.3.2.2. graphInitialized = true
 - 7.3.2.3. Вывести сообщение об успешной генерации
 - 7.3.2.4. Показать граф
- 7.4. Иначе если choice == 2:
 - 7.4.1. Если graphInitialized: показать матрицу смежности
 - 7.4.2. Иначе: вывести сообщение об ошибке
- 7.5. Иначе если choice == 3:
 - 7.5.1. Если graphInitialized:
 - 7.5.1.1. Запросить startVertex
 - 7.5.1.2. Если startVertex корректна: выполнить BFSFromSpecificVertex
 - 7.5.1.3. Иначе: вывести сообщение об ошибке
 - 7.5.2. Иначе: вывести сообщение об ошибке
- 7.6. Иначе если choice == 4:
 - 7.6.1. Если graphInitialized: найти все компоненты связности
 - 7.6.2. Иначе: вывести сообщение об ошибке
- 7.7. Иначе если choice == 5:
 - 7.7.1. Если graphInitialized: сгенерировать новый граф, показать его
 - 7.7.2. Иначе: вывести сообщение об ошибке
- 7.8. Иначе если choice == 6:
 - 7.8.1. Если graphInitialized: сохранить граф в файл
 - 7.8.2. Иначе: вывести сообщение об ошибке
- 7.9. Иначе если choice == 7:
 - 7.9.1. Если загрузка успешна:
 - 7.9.1.1. graphInitialized = true

7.9.1.2. Показать граф

7.10. Иначе если choice == 8:

7.10.1. Если graphInitialized:

7.10.1.1. Запросить подтверждение сохранения

7.10.1.2. Если подтверждено: сохранить граф

7.10.2. Выйти из цикла

7.11. Иначе: вывести сообщение о неверном выборе

8. Завершить программу

Работа с очередью:

initQueue(очередь q):

1. q.front = -1

2. q.rear = -1

isEmpty(очередь q):

1. Вернуть (q.front == -1)

enqueue(очередь q, значение value):

1. Если q.rear == MAX_VERTICES-1: вывести "Очередь переполнена"

2. Иначе:

2.1. Если q.front == -1: q.front = 0

2.2. q.rear = q.rear + 1

2.3. q.items[q.rear] = value

dequeue(очередь q):

1. Если isEmpty(q):

1.1. Вывести "Очередь пуста"

1.2. Вернуть -1

2. Иначе:

2.1. item = q.items[q.front]

2.2. q.front = q.front + 1

2.3. Если q.front > q.rear: q.front = q.rear = -1

2.4. Вернуть item

На рисунке изображён вывод главного меню

```
=====
      АЛГОРИТМ BFS ДЛЯ ОРГРАФА
=====
1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф
6. Сохранить граф в файл 'Grafi.txt'
7. Загрузить граф из файла 'Grafi.txt'
8. Выход
=====
Выберите действие:
```

Рисунок 1-Главное меню

На рисунке изображена генерация случайного графа с выводом типа СВЯЗНОСТИ

```
=== Матрица смежности орграфа (6 вершин) ===
      0  1  2  3  4  5
0:    0  1  1  0  1  1
1:    0  0  1  1  1  1
2:    1  1  0  0  1  1
3:    0  0  1  0  1  1
4:    0  0  1  0  0  0
5:    1  1  0  1  1  0
Всего ребер: 20

=== АНАЛИЗ СВЯЗНОСТИ ОРГРАФА ===
Орграф связный
Орграф сильно связан
```

Рисунок 2- Генерация случайного графа с выводом типа связности

На рисунке изображён вывод компоненты связности орграфа

```
=== Все компоненты связности орграфа ===
Компонента 1: 0 1 2 4 5 3
Всего компонент связности: 1
```

Рисунок 3-Вывод компоненты связности орграфа

На рисунке изображено сохранение графа в файл

```
=====
                АЛГОРИТМ BFS ДЛЯ ОРГРАФА
=====
1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф
6. Сохранить граф в файл 'Graf1.txt'
7. Загрузить граф из файла 'Graf1.txt'
8. Определить тип связности графа
9. Выход
=====
Выберите действие: 6
Граф успешно сохранен в файл 'Graf1.txt'
```

Рисунок 4- Сохранение графа в файл

5.Тестирование

Тестирование проводилось по следующим направлениям:

- 1.Функциональное тестирование: проверка корректности работы всех функций
- 2.Тестирование граничных условий: работа с минимальными и максимальными размерами графов
- 3.Тестирование обработки ошибок: реакция на некорректный ввод

На рисунке изображен вывод меню и орграфа

```
=====
                АЛГОРИТМ BFS ДЛЯ ОРГРАФА
=====
1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф
6. Сохранить граф в файл 'Grafi.txt'
7. Загрузить граф из файла 'Grafi.txt'
8. Определить тип связности графа
9. Выход
=====
Выберите действие: 1
Введите количество вершин графа (1-50): 6
Случайный орграф успешно сгенерирован!

=== Матрица смежности орграфа (6 вершин) ===
      0  1  2  3  4  5
0:    0  1  1  0  0  1
1:    0  0  0  1  0  1
2:    0  1  0  1  1  1
3:    1  1  0  0  0  0
4:    0  1  0  0  0  0
5:    0  1  0  0  0  0
Всего ребер: 13

=== АНАЛИЗ СВЯЗНОСТИ ОРГРАФА ===
Орграф связный
Орграф сильно связан
```

Рисунок 5 - вывод меню и орграфа

На рисунке изображена работа с минимальным размером орграфа

```
=== Матрица смежности орграфа (2 вершин) ===
  0  1
0:  0  1
1:  1  0
Всего ребер: 2

=== АНАЛИЗ СВЯЗНОСТИ ОРГРАФА ===
Орграф связный
Орграф сильно связан

=====
      АЛГОРИТМ BFS ДЛЯ ОРГРАФА
=====
1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф
6. Сохранить граф в файл 'Grafi.txt'
7. Загрузить граф из файла 'Grafi.txt'
8. Определить тип связности графа
9. Выход
=====
Выберите действие: 3
Введите стартовую вершину для BFS (0-1): 0

=== BFS обход начиная с вершины 0 ===
Порядок обхода BFS начиная с вершины 0: 0 1
Недостижимые вершины: все вершины достижимы

=====
      АЛГОРИТМ BFS ДЛЯ ОРГРАФА
=====
1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф
6. Сохранить граф в файл 'Grafi.txt'
7. Загрузить граф из файла 'Grafi.txt'
8. Определить тип связности графа
9. Выход
=====
Выберите действие: 4

=== Все компоненты связности орграфа ===
Компонента 1: 0 1
Всего компонент связности: 1
```

Рисунок 6 - работа с минимальным размером орграфа

На рисунке изображена работа с максимальным размером орграфа

```
31: 0 1 0 1 0 0 0 0 1 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 1 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1
32: 0 1 1 1 0 1 0 1 0 0 0 1 0 0 0 1 1 1 1 1 0 0 0 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0
33: 0 1 0 0 1 0 1 0 0 1 1 1 1 1 0 0 1 0 0 0 1 1 1 0 0 0 0 1 1 1 0 0 0 1 1 1 0 0 0 1 1 1 0 0 0 1 0 0 0 0
34: 0 1 1 0 0 1 1 0 0 1 1 0 0 0 1 1 1 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 1 1 1 1 0 0 1 1 0 0 0 1 1 0 0 1 1 0
35: 0 0 0 1 1 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 1 1 1 1 1 0 0 0 1 1 0 1 0 1 0 1 0 1 1 1 1
36: 0 1 0 0 1 1 0 0 0 1 1 1 1 1 0 0 1 0 0 0 1 1 0 0 0 1 1 0 0 0 1 1 1 1 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 1
37: 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1 1 0 1 0 0 0 1 1 1 1 1 0 0 1 1 1 1 0 0 0 1 0 0 0 0 1 0 0 0 0 1 1 1 0 1
38: 0 1 0 0 0 0 0 1 0 0 1 1 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1 1 1 1 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 0 0
39: 1 1 0 0 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 0 0 0 1 1 1 0 0 0 1 1 1 0 0 1 0 0 1 1 0 1 1 0 1 1 0 1 0 0 0
40: 0 0 1 1 1 1 0 0 0 1 0 1 0 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 0 1 0 0 0 1 0 1 1 1 1 0 0 0 1 0 0 1 1 1 1 1
41: 1 1 0 0 0 1 1 0 1 0 0 1 1 1 0 0 0 1 1 0 0 0 1 0 0 0 0 0 0 0 1 1 1 0 0 0 1 1 0 0 1 0 0 1 1 1 0 1 1 0
42: 1 1 1 0 0 0 0 1 1 1 1 0 1 1 1 1 1 1 0 0 1 1 0 0 0 1 1 0 0 0 1 1 1 1 0 0 0 1 1 0 0 0 1 0 0 0 0 1 1 0 0 1
43: 1 0 1 1 0 1 0 0 0 1 0 1 0 0 0 1 1 1 1 0 1 0 0 0 0 1 0 0 0 1 0 0 0 0 0 1 1 0 1 0 1 1 0 0 0 0 1 1 0 1
44: 0 1 1 1 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 0 1 1 1 1 1 0 1 1 0 1 0 1 0 1 0 1 0 0 1 0 0 0 1 1 1 0 1 0 0 1 0
45: 1 1 0 1 0 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 1 0 0 0 0 1 1 1 1 0 0 0 0 1 0 0 0 0 1 0 0 1 1 1 0 1 1 0 0 0
46: 1 1 1 1 0 0 0 1 1 1 0 1 1 0 1 0 1 0 0 1 1 0 1 0 0 1 1 1 1 0 0 1 1 1 1 0 1 1 1 0 1 0 1 0 1 0 1 0 1 1 0
47: 1 0 0 1 0 0 1 0 1 0 1 0 0 0 1 0 1 1 1 0 0 0 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 1 0 0 0 0 1 0 0
48: 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0 0 0 1 1 0 1 1 1 1 0 0 0 1 0 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 0 0
49: 1 0 0 1 1 0 1 0 0 1 1 1 1 1 0 1 0 1 1 1 0 0 0 0 1 1 0 0 0 1 0 0 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 0
всего ребер: 1241
--- АНАЛИЗ СВЯЗНОСТИ ОРГРАФА ---
Орграф связный
Орграф сильно связан
=====
АЛГОРИТМ BFS ДЛЯ ОРГРАФА
=====
1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф
6. Сохранить граф в файл "Grafi.txt"
7. Загрузить граф из файла "Grafi.txt"
8. Определить тип связности графа
9. Выход
=====
Выберите действие: 3
Введите стартовую вершину для BFS (0-49): 5
--- BFS обход начиная с вершины 5 ---
Порядок обхода BFS начиная с вершины 5: 5 0 1 3 7 8 10 15 17 18 19 21 23 25 26 27 28 29 30 31 34 36 38 39 40 42 43 45 47 49 2 6 9 11 12 13 14 32 33 37 41 22 44 48 35 4 20 24 16 46
Недостижимые вершины: все вершины достижимы
=====
АЛГОРИТМ BFS ДЛЯ ОРГРАФА
=====
1. Ввести количество вершин и сгенерировать случайный граф
2. Показать матрицу смежности
3. Выполнить BFS с заданной стартовой вершиной
4. Найти все компоненты связности
5. Сгенерировать новый случайный граф
6. Сохранить граф в файл "Grafi.txt"
7. Загрузить граф из файла "Grafi.txt"
8. Определить тип связности графа
9. Выход
=====
Выберите действие: 4
--- Все компоненты связности орграфа ---
Компоненты 1: 0 1 2 3 6 7 8 9 11 12 13 14 15 17 18 23 25 27 28 30 32 33 34 36 37 38 39 41 43 49 10 19 21 22 26 29 31 44 45 48 4 5 16 35 40 46 47 42 20 24
всего компонент связности: 1
```

Рисунок 7-работа с максимальным размером орграфа

Таблица 1 – Описание поведения программы при тестировании

Описание теста	Ожидаемый результат	Полученный результат
Запуск программы	Отображение главного меню	Верно
Выбор генерации матрицы	Вывод сообщения о количестве вершин в орграфе	Верно
Выбор вывода матрицы из памяти	Вывод меню с видами орграфа	Верно
Выбор слабо связного орграфа	Вывод слабо связного орграфа и компоненты его сильной связности	Верно
Выбор генерации матрицы с 6 вершинами	Вывод матрицы смежности с 6 вершинами. Вывод вида графа и степень его связности, а также компоненты его сильной связности	Верно

В результате тестирования было выявлено, что программа успешно проверяет данные на соответствие необходимым требованиям.

6. Ручной расчёт задачи

На рисунке изображен ручной расчёт

```
=== Матрица смежности орграфа (6 вершин) ===
      0  1  2  3  4  5
0:    0  1  0  0  1  0
1:    0  0  1  0  0  0
2:    1  0  0  1  0  0
3:    0  1  1  0  1  0
4:    1  0  0  0  0  0
5:    0  0  0  0  1  0
Всего ребер: 10
```

Рисунок 8-ручной расчёт

Посещаем все вершины, начиная с 0:

Очередь: [0]

Посещённые: {0}

Из 0: соседи (из 0 выходят 1,4; и входят: $2 \rightarrow 0$, $4 \rightarrow 0$)

Значит соседи 0: {1, 4, 2} (так как 2 связан с 0 через ребро $2 \rightarrow 0$).

Порядок BFS:

Шаг 1: Из очереди берём 0, добавляем 1, 4, 2 в очередь (если не посещены).

Посещаем 1, 4, 2, кладём их в очередь: очередь [1, 4, 2]

Посещённые: {0, 1, 4, 2}

Шаг 2: Берём 1, её соседи: из $1 \rightarrow 2$, и входят: $0 \rightarrow 1$, $3 \rightarrow 1$

Соседи {0, 2, 3}

0 и 2 уже посещены, добавляем 3.

Очередь [4, 2, 3], посещённые {0,1,4,2,3}

Шаг 3: Берём 4, соседи: из $4 \rightarrow 0$, входят: $0 \rightarrow 4$, $3 \rightarrow 4$, $5 \rightarrow 4$

Соседи {0,3,5}

0 и 3 посещены, добавляем 5.

Очередь [2,3,5], посещённые {0,1,4,2,3,5}

Шаг 4: Берём 2, соседи: из 2 $\rightarrow$ 0, 3, входят: 1 $\rightarrow$ 2, 3 $\rightarrow$ 2

Соседи {0,3,1} — все посещены.

Очередь [3,5]

Шаг 5: Берём 3, соседи: из 3 $\rightarrow$ 1,2,4, входят: 2 $\rightarrow$ 3, 1 $\rightarrow$ 3, 4 $\rightarrow$ 3

{1,2,4} — все посещены.

Очередь [5]

Шаг 6: Берём 5 соседи: из 5 $\rightarrow$ 4, входит: нет входящих кроме 4 $\rightarrow$ 5

Сосед {4} — посещён.

Очередь пуста.

Все вершины посещены $\Rightarrow$ одна компонента слабой связности.

Заключение

В ходе выполнения курсового проекта была разработана программа на языке Си, реализующая алгоритм поиска в ширину для выделения компонент связности ориентированных графов.

Основные достижения:

1.Реализация алгоритма BFS: создана эффективная реализация поиска в ширину с использованием очереди

2.Выделение компонент связности: разработан алгоритм для нахождения всех компонент связности орграфа

3.Определение типа связности: программа определяет, является ли граф сильно связным, слабо связным или несвязным

4.Работа с файлами: реализованы функции сохранения и загрузки графов

5.Визуализация: добавлена возможность экспорта графов в DOT формат для визуализации в Graphviz

6.Пользовательский интерфейс: создан удобный консольный интерфейс с поддержкой русского языка

Список литературы

- 1.Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – М.: Вильямс, 2013. – 1296 с.
- 2.Ахо А., Хопкрофт Дж., Ульман Дж. Построение и анализ вычислительных алгоритмов. – М.: Мир, 1979. – 536 с.
- 3.Оре О. Теория графов. – М.: Наука, 1980. – 336 с.
- 4.Харари Ф. Теория графов. – М.: Мир, 1973. – 300 с.
- 5.Седжвик Р. Фундаментальные алгоритмы на C++. Часть 5. Алгоритмы на графах. – М.: Диалектика, 2002. – 496 с.
- 6.Вирт Н. Алгоритмы и структуры данных. – М.: Мир, 1989. – 360 с.

ПриложениеА. Листингпрограммы.

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <locale.h>
```

```
#include <stdbool.h>
```

```
#include <time.h>
```

```
#include <windows.h>
```

```
#define MAX_VERTICES 50
```

```
#define FILENAME "Graf.txt"
```

```
typedef struct {
```

```
    int adj[MAX_VERTICES][MAX_VERTICES];
```

```
    int vertices;
```

```
} Graph;
```

```
typedef struct {
```

```
    int items[MAX_VERTICES];
```

```
    int front;
```

```
    int rear;
```

```
} Queue;
```

```
void initGraph(Graph* g, int vertices) {
```

```
    g->vertices = vertices;
```

```
    for (int i = 0; i < vertices; i++) {
```

```

        for (int j = 0; j < vertices; j++) {
            g->adj[i][j] = 0;
        }
    }
}

void addDirectedEdge(Graph* g, int src, int dest) {
    if (src >= 0 && src < g->vertices && dest >= 0 && dest < g->vertices) {
        g->adj[src][dest] = 1;
    }
}

void saveGraphToFile(Graph* g) {
    FILE* file = fopen(FILENAME, "w");
    if (file == NULL) {
        printf("Ошибка при открытии файла для записи!\n");
        return;
    }

    fprintf(file, "%d\n", g->vertices);

    for (int i = 0; i < g->vertices; i++) {
        for (int j = 0; j < g->vertices; j++) {
            fprintf(file, "%d ", g->adj[i][j]);
        }
        fprintf(file, "\n");
    }
}

```

```

fclose(file);

printf("Граф успешно сохранен в файл '%s'\n", FILENAME);
}

int loadGraphFromFile(Graph* g) {
    FILE* file = fopen(FILENAME, "r");

    if (file == NULL) {
        printf("Файл '%s' не найден. Создайте новый граф.\n", FILENAME);
        return 0;
    }

    if (fscanf(file, "%d", &g->vertices) != 1) {
        printf("Ошибка при чтении файла!\n");
        fclose(file);
        return 0;
    }

    if (g->vertices <= 0 || g->vertices > MAX_VERTICES) {
        printf("Некорректное количество вершин в файле!\n");
        fclose(file);
        return 0;
    }

    for (int i = 0; i < g->vertices; i++) {
        for (int j = 0; j < g->vertices; j++) {
            if (fscanf(file, "%d", &g->adj[i][j]) != 1) {
                printf("Ошибка при чтении матрицы смежности!\n");
                fclose(file);
            }
        }
    }
}

```

```

        return 0;
    }
}

fclose(file);

printf("Граф успешно загружен из файла '%s'\n", FILENAME);

return 1;
}

void generateRandomGraph(Graph* g, int vertices) {
    initGraph(g, vertices);
    srand(time(NULL));

    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            if (i != j && rand() % 2 == 0) {
                g->adj[i][j] = 1;
            }
        }
    }
}

void initQueue(Queue* q) { q->front = -1; q->rear = -1; }

bool isEmpty(Queue* q) { return q->front == -1; }

void enqueue(Queue* q, int value) {
    if (q->rear == MAX_VERTICES - 1) return;

```

```

    if (q->front == -1) q->front = 0;

    q->rear++;

    q->items[q->rear] = value;
}

int dequeue(Queue* q) {
    if (isEmpty(q)) return -1;

    int item = q->items[q->front];

    q->front++;

    if (q->front > q->rear) q->front = q->rear = -1;

    return item;
}

void BFS(Graph* g, int start, bool visited[]) {
    Queue q;

    initQueue(&q);

    visited[start] = true;

    enqueue(&q, start);

    while (!isEmpty(&q)) {
        int v = dequeue(&q);

        for (int i = 0; i < g->vertices; i++) {
            if (g->adj[v][i] && !visited[i]) {
                visited[i] = true;

                enqueue(&q, i);
            }
        }
    }
}

```

```
}
```

```
int isStronglyConnected(Graph* g) {  
    for (int i = 0; i < g->vertices; i++) {  
        bool visited[MAX_VERTICES] = {false};  
        BFS(g, i, visited);  
        for (int j = 0; j < g->vertices; j++) {  
            if (!visited[j]) return 0;  
        }  
    }  
    return 1;  
}
```

```
int isWeaklyConnected(Graph* g) {  
    // Создаем неориентированный граф  
    bool undirected[MAX_VERTICES][MAX_VERTICES] = {false};  
    for (int i = 0; i < g->vertices; i++) {  
        for (int j = 0; j < g->vertices; j++) {  
            if (g->adj[i][j]) {  
                undirected[i][j] = true;  
                undirected[j][i] = true;  
            }  
        }  
    }  
}
```

```
    bool visited[MAX_VERTICES] = {false};  
    Queue q;  
    initQueue(&q);
```

```

visited[0] = true;
enqueue(&q, 0);

while (!isEmpty(&q)) {
    int v = dequeue(&q);
    for (int i = 0; i < g->vertices; i++) {
        if (undirected[v][i] && !visited[i]) {
            visited[i] = true;
            enqueue(&q, i);
        }
    }
}

for (int i = 0; i < g->vertices; i++) {
    if (!visited[i]) return 0;
}
return 1;
}

void checkConnectivity(Graph* g) {
    printf("\n=== АНАЛИЗ СВЯЗНОСТИ ОРГРАФА ===\n");

    if (isWeaklyConnected(g)) {
        printf("Орграф связный\n");
        if (isStronglyConnected(g)) {
            printf("Орграф сильно связан\n");
        }
        else {

```

```

printf("Орграфслабосвязный\n");
    }
}
else {
printf("Орграфнесвязный\n");
    }
}

void BFSWithDisplay(Graph* g, int startVertex) {
    bool visited[MAX_VERTICES] = {false};
    Queue q;
    initQueue(&q);

    printf("ПорядокобходаBFSначинаясвершины %d: ", startVertex);
    visited[startVertex] = true;
    enqueue(&q, startVertex);

    while (!isEmpty(&q)) {
        int v = dequeue(&q);
        printf("%d ", v);
        for (int i = 0; i < g->vertices; i++) {
            if (g->adj[v][i] && !visited[i]) {
                visited[i] = true;
                enqueue(&q, i);
            }
        }
    }

    printf("\n");
}

```

```
printf("Недостижимые вершины: ");
```

```
bool allReachable = true;
```

```
for (int i = 0; i < g->vertices; i++) {
```

```
    if (!visited[i]) {
```

```
printf("%d ", i);
```

```
allReachable = false;
```

```
    }
```

```
}
```

```
if (allReachable) {
```

```
printf("все вершины достижимы");
```

```
}
```

```
printf("\n");
```

```
}
```

```
void displayGraph(Graph* g) {
```

```
printf("\n=== Матрица смежности орграфа (%d вершин) ===\n", g->vertices);
```

```
printf("  ");
```

```
for (int i = 0; i < g->vertices; i++) {
```

```
printf("%2d ", i);
```

```
}
```

```
printf("\n");
```

```
for (int i = 0; i < g->vertices; i++) {
```

```
printf("%2d: ", i);
```

```
for (int j = 0; j < g->vertices; j++) {
```

```
printf("%2d ", g->adj[i][j]);
```

```
}
```

```

printf("\n");

}

int edgeCount = 0;

for (int i = 0; i < g->vertices; i++) {
    for (int j = 0; j < g->vertices; j++) {
        if (g->adj[i][j] == 1) {
            edgeCount++;
        }
    }
}

printf("Всего ребер: %d\n", edgeCount);
}

void displayMenu() {
    printf("\n===== \n");
    printf("    АЛГОРИТМ BFS ДЛЯ ОРГРАФА\n");
    printf("===== \n");
    printf("1. Ввести количество вершин и сгенерировать случайный граф\n");
    printf("2. Показать матрицу смежности\n");
    printf("3. Выполнить BFS с заданной стартовой вершиной\n");
    printf("4. Найти все компоненты связности\n");
    printf("5. Сгенерировать новый случайный граф\n");
    printf("6. Сохранить граф в файл '%s'\n", FILENAME);
    printf("7. Загрузить граф из файла '%s'\n", FILENAME);
    printf("8. Определить тип связности графа\n");
    printf("9. Выход\n");
    printf("===== \n");
}

```

```

printf("Выберите действие: ");

}

void findAllComponents(Graph* g) {
    bool visited[MAX_VERTICES] = {false};
    int count = 0;

    printf("\n=== Все компоненты связности орграфа ===\n");
    for (int i = 0; i < g->vertices; i++) {
        if (!visited[i]) {
            printf("Компонента %d: ", ++count);

            Queue q;
            initQueue(&q);
            visited[i] = true;
            enqueue(&q, i);

            while (!isEmpty(&q)) {
                int v = dequeue(&q);
                printf("%d ", v);

                for (int j = 0; j < g->vertices; j++) {
                    if (g->adj[v][j] && !visited[j]) {
                        visited[j] = true;
                        enqueue(&q, j);
                    }
                }
            }

            printf("\n");
        }
    }
}

```

```

    }

}

printf("Всегокомпонентсвязности: %d\n\n", count);

}

int main() {

SetConsoleCP(1251);

SetConsoleOutputCP(1251);

setlocale(LC_ALL, "Russian");

srand(time(NULL));

    Graph graph;

    int vertices = 0;

    int choice;

    bool graphInitialized = false;

printf("=== РЕАЛИЗАЦИЯ АЛГОРИТМА BFS ДЛЯ ОРГРАФА ===\n");

printf("Алгоритм выделения компонент связности орграфа\n");

printf("с использованием поиска в ширину (BFS)\n");

printf("Работа с файлом: %s\n\n", FILENAME);

printf("Попытка загрузки графа из файла '%s'...\n", FILENAME);

if (loadGraphFromFile(&graph)) {

graphInitialized = true;

    vertices = graph.vertices;

}

```

```

do {
displayMenu();
scanf("%d", &choice);

    switch (choice) {
case 1: {
printf("Введите количество вершин графа (1-%d): ", MAX_VERTICES);
scanf("%d", &vertices);

        if (vertices < 1 || vertices > MAX_VERTICES) {
printf("Ошибка: количество вершин должно быть от 1 до %d\n", MAX_VERTICES);
break;
        }

generateRandomGraph(&graph, vertices);
graphInitialized = true;
printf("Случайный орграф успешно сгенерирован!\n");
displayGraph(&graph);
checkConnectivity(&graph);
        break;
    }

case 2:
if(!graphInitialized) {
printf("Ошибка: граф не инициализирован. Сначала выберите пункт 1 или 7\n");
}

        else {
displayGraph(&graph);

```

```

    }

    break;

case 3:

    if (!graphInitialized) {
printf("Ошибка: граф не инициализирован. Сначала выберите пункт 1 или 7\n");

    }

else {

int startVertex;

printf("Введите стартовую вершину для BFS (0-%d): ", vertices - 1);

scanf("%d", &startVertex);

    if (startVertex < 0 || startVertex >= vertices) {

printf("Ошибка: вершина %d не существует в графе (вершины от 0 до %d)\n",
startVertex, vertices - 1);

    }

else {

printf("\n=== BFS обход начиная с вершины %d ===\n", startVertex);

BFSWithDisplay(&graph, startVertex);

    }

}

break;

case 4:

    if (!graphInitialized) {

printf("Ошибка: граф не инициализирован. Сначала выберите пункт 1 или 7\n");

    }

else {

findAllComponents(&graph);

```

```

    }

    break;

case 5:

    if (!graphInitialized) {
printf("Ошибка: граф не инициализирован. Сначала выберите пункт 1 или 7\n");
    }

    else {
printf("Генерация нового случайного графа...\n");
generateRandomGraph(&graph, vertices);
printf("Новый граф успешно сгенерирован!\n");
displayGraph(&graph);
checkConnectivity(&graph);
    }

    break;

case 6:

if(!graphInitialized) {
printf("Ошибка: граф не инициализирован. Сначала создайте или загрузите граф\n");
}

    else {
saveGraphToFile(&graph);
    }

    break;

case 7:

    if (loadGraphFromFile(&graph)) {
graphInitialized = true;

```

```

        vertices = graph.vertices;

displayGraph(&graph);
checkConnectivity(&graph);
    }

    break;

case 8:

    if (!graphInitialized) {
printf("Ошибка: граф не инициализирован. Сначала создайте и загрузите граф\n");
    }

    else {
checkConnectivity(&graph);
    }

    break;

case 9:

printf("Выход из программы...\n");

if (graphInitialized) {

    char saveChoice;

printf("Сохранить граф перед выходом? (y/n): ");

scanf(" %c", &saveChoice);

    if (saveChoice == 'y' || saveChoice == 'Y') {
saveGraphToFile(&graph);
    }

}

break;

default:

```

```
printf("Ошибка: неверный выбор. Попробуйте снова.\n");  
    }  
  
} while (choice != 9);  
  
return 0;  
}
```